CS 382 Network Centric Computing

Assignment 4 - Peer to Peer (P2P) File Sharing System

Network Centric Computing - Spring '26

Lead TAs: Zaeem Mohtashim, Ammara Haroon, Anas Asim, Izza Shahid

Due Date: 11:55pm, 10th May, 2026

Please note that final day of submissions is 13th May 2026.

Contents

Disclaimer on Plagiarism and AI Tools

Academic integrity is paramount. Any form of plagiarism, including but not limited to copying code from online sources, unauthorized collaboration, or using AI-assisted tools such as GitHub Copilot, ChatGPT, or similar, is strictly prohibited.

All submissions will be analyzed using **MOSS** and other plagiarism detection tools. If plagiarism or unauthorized AI-assisted work is detected, the case will be forwarded to the **Disciplinary Committee (DC)** for review and appropriate action, which may include academic penalties or further disciplinary measures.

"AI wrote this for me" is not a valid justification. Submissions should reflect **your own** understanding, problem-solving ability, and effort. While AI tools can be useful for learning, relying on them to generate solutions undermines the purpose of the assignment and will be treated as academic misconduct.

Ensure that all work submitted is your **own** and adheres to academic integrity policies. If you have any doubts about what constitutes plagiarism or unauthorized AI use, seek clarification before submission.

1 Assignment Overview

The key purpose of this assignment is to familiarize you with the concept of consistent hashing, which is widely used in distributed systems. You will be building a peer-peer file sharing system that uses Distributed Hash Table, leverages consistent hashing, and is resilient to node failures.

1.1 Queries

You should post any assignment related queries ONLY on Slack. Please do not email the course staff regarding any assignment related queries. This will ensure that everyone benefits from the answers to your queries.

1.2 Grade Distribution

This assignment is worth **7.5%** of your final grade.

It will be graded out of 45 points. The distribution of points is as follows:

- **Initialization** (1 point)
- **Join** (9 points)
  - Corner case of 1 node (1 point)
  - Corner case of 2 nodes (2 points)
  - General case (5 points)
- **Put and Get** (10 points)
  - Put Test (7 points)
  - Get Test (3 points)
- **File Transfer on Join** (5 points)
- **Leave** (5 points)
  - Updating successor and predecessors of affected nodes (3 points)
  - Files transferred correctly (2 points)
- **Failure Tolerance** (10 points)
  - Updating successor and predecessors of affected nodes (3 points)
  - Files recovered on correct nodes (7 points)
- **Code Quality** (5 points)
  - Following good code practices as described in section 1.4

The `kill` function has only been provided for debugging. Submissions that implement this function (i.e. include in it more than just print statements) will not be accepted.

1.3 Submission Instructions

You should submit your assignment on LMS. You should submit a zip file containing the following:

- `DHT.py`

The zip file should be named `<RollNumber>.zip`. For example, if your roll number is `24100027`, your submission file should be named `24100027.zip`.

You should not submit any other files. You will be penalized (**2% of the assignment grade**) if you do not follow the submission guidelines.

1.4 Code Quality

Your code will be evaluated using automated tools as well as a manual review based on the following criteria:

- **Maintainability, Modularity, and Complexity:** Your code will be analyzed using tools like Radon and Pylint to assess maintainability, modularity, and cyclomatic complexity. The grading will follow the detailed breakdown provided in Section 7.
- **Naming Conventions:** You are expected to follow PEP 8 naming conventions. You can find the standards here. We will specifically look at:
  - **Variables, Functions, and Methods:** Use lowercase letters and separate words with underscores. For example, `my_variable`, `calculate_sum()`.
  - **Constants:** Use uppercase letters and separate words with underscores. For example, `MAX_SIZE`, `PI`.
  - **Classes:** Use CamelCase (capitalize the first letter of each word without underscores). For example, `MyClass`, `CarModel`.
- **Code Documentation and Comments:** Your code should be well-documented, following the comment ratio guidelines. The comment percentage will be evaluated as described in Section 7, ensuring that your code is understandable to others.
- **Code Modularity and Reusability:** Your implementation should not be monolithic; instead, break down your code into smaller reusable functions. This will be assessed as part of the modularity score in the automated quality check.
- **Readability and Formatting:** Your code should be readable with meaningful variable and function names. Additionally, proper indentation and spacing should be maintained to improve readability.

The final quality score will be calculated using automated scripts. Ensure that your code adheres to these best practices, as they are essential for writing maintainable and efficient software.

2 Introduction

In this assignment, you will build a peer-peer file sharing system. Your system will be a Distributed Hash Table (DHT) based key-value storage system that has only two basic operations; `put()` and `get()`. The `put()` operation takes as input a filename, evaluates its key using some hash function, and places the file on one of the nodes in the distributed system. Whereas, the `get()` function takes as input a filename, finds the appropriate node that can potentially have

that file and returns the file if it exists on that node. We will make this system failure tolerant i.e. it should not lose any data in case of failure of a node.

You will be building the DHT using consistent hashing. Consistent hashing is a scheme for distributing key-value pairs, such that distribution does not depend upon number of nodes in the DHT. This allows to scale the size of DHT easily i.e. addition or removal of new nodes in the system is not a costly operation.

3 Starter Code

You have been provided with the following files in the Code directory:

```
PA4
|-- docker-compose.yaml
|-- Code
|   |-- check.py
|   |-- DHT.py
|   |-- requirements.txt
|   |-- ReviewCodeQuality.py
|   |-- run_multiple_tests.py
|   `-- test
`-- Manual
    `-- PA4.pdf
```

You will write code in the `DHT.py` file ONLY.

Please note that `check.py` is for testing. You may modify it but we will only run the file we have provided.

4 DHT.py

We will test your assignment using the following API.

- `Node()` : Initialization of Node, you may maintain any state you need here. Following variables which are already declared should not be renamed and should be set appropriately since testing will take place through them. Some of them have already been set to appropriate values.
  - `host`: Save hostname of Node e.g. localhost
  - `port`: Save port of Node e.g. 8000
  - `M`: hash value's number of bits
  - `N`: Size of the ring
  - `key`: it is the hashed value of Node
  - `successor`: Next node's address (host, port)
  - `predecessor`: Previous node's address (host, port)
  - `files`: Files mapped to this node
  - `backup_files`: Files saved as back up on this node
- `join()` : This function handles the logic for a new node joining the DHT; this function should update node's successor and predecessor. It should also trigger the update of affected nodes'

(i.e. successor node and predecessor node) successor and predecessor too. Finally, the node should also get its share of files.

- `leave()` : Calling `leave` should have the following effects: removal of node from the ring and transfer of files to the appropriate node.
- `put()` : This function should handle placement of file on appropriate node; save the files in the directory for the node.
- `get()` : This function is responsible for looking up and getting a file from the network.

Some functions have already been provided to you as utility functions. You can also create more helper functions.

1. `hasher()`: Hashes a string to an M bits long number. You should use it as follows:
   - For a node: `hasher(node.host+str(node.port))`
   - For a file: `hasher(filename)`
2. `listener()`: This function listens for new incoming connections. For every new connection, it spins a new thread `handle_connection` to handle that connection. You may write any necessary logic for any connection there.
3. `send_file()`: You can use this function to send a file over the socket to another node.
4. `receive_file()`: This function can be used to receive files over the socket from other nodes. Both these functions have the following arguments:
   - `soc`: A TCP socket object.
   - `file_name`: File's name along with complete path e.g. `CS382/PA4/file.py`

5 Implementation Instructions

Although assignment can be done in any order, we suggest you follow the following pattern as this order of checking is followed in testing as well. You can refer to the marking details above.

Note: You are required to use sockets for any communication between the nodes i.e. you should not access another node's variables or functions directly.

5.1 Initialization

When a new node is created, its successor and predecessor do not point to any other node as it does not know any other node yet. In this case both these pointers should point to the node itself. `successor` and `predecessor` both store a tuple of host and port e.g. `('localhost', 20007)`.

5.2 Join

Before you implement `join`, `put` or `get`; it is a good idea to implement a look-up function. This look-up function should be able to find the node responsible for a given key. Benefit of doing this is that you can reuse this function in `join`, `put` and `get`.

Let's come to `join` now. Join function takes as input the address (host, port) of a node (`joiningAddr`) already present in the DHT. You will connect with this node and ask it to do a lookup for the new node's successor and update the new node's successor based on the response. You should also write the logic of updating the predecessor. This usually happens

through pinging; a node keeps pinging (after around 0.5 seconds) its successor to know whether it is still its successor's predecessor. If the successor node has updated its predecessor, this node should update its successor to the current successor's predecessor. The joining process is explained in Figure 1.

You should pay special attention to corner cases such as when there is only one node in the system; in this case, instead of the joining address, an empty string will be passed. Similarly, when there are only two nodes, they will be each other's successor and predecessor.

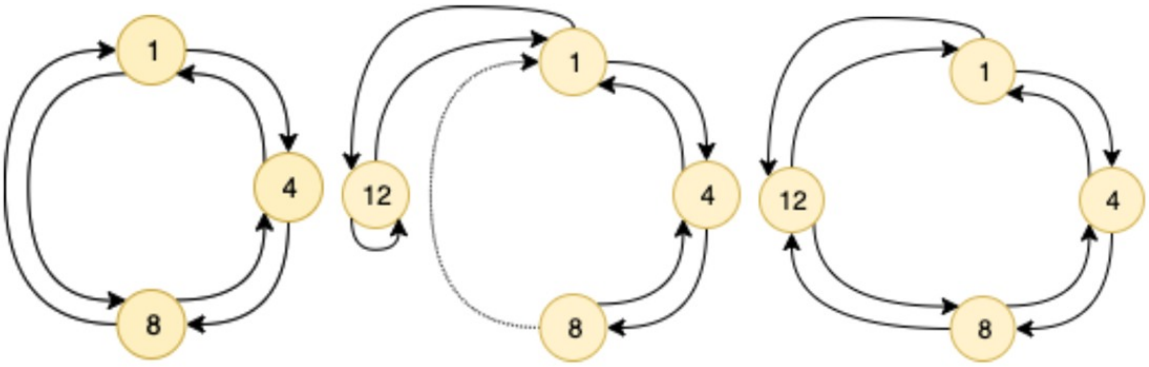

Figure 1: **i)** A DHT with three nodes, clockwise arrows are for successors and anti-clockwise are for predecessors. **ii)** A new node 12 is added, it finds its position and updates its successor to point at 1, 1 points its predecessor at 12. **iii)** 8 asks 1 for its predecessor and gets 12, it updates its successor to 12, 12 updates its predecessor to 8

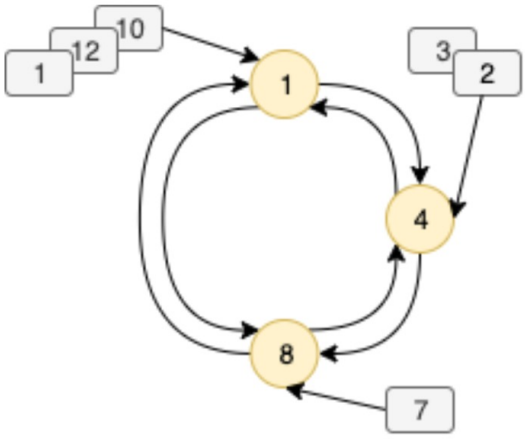

Figure 2: Placement of files on a DHT, every node is responsible for space between itself and its predecessor

5.3 Put and Get

When `put` is called on a node with a file, it finds the node responsible for that file and sends the file to that node. You have to use `send_file` and `receive_file` functions here. You should store the file in the directory assigned to that node given by node's host and port as `host_port` e.g. `localhost_20007`. This directory is already made in the provided starter code. Similarly, the `get` method should again first find the node responsible for the file and then retrieve the

file from that node. If the file exists you should return the filename, if it does not you should return `None`.

Important Instruction

Again, you have to use the `send_file` and `receive_file` functions. This time you should get the file from the node and save it in a directory called `"test"` specifically, and not the current directory.

Every node is responsible for the space between itself and its predecessor. Refer to Figure 2 for a detailed explanation.

5.4 File Transfer on Join

When a new node joins the network, it should get its share of files i.e. the files that hash to its space. Think about what file will come in share of a new node. After transferring the file, the new node will be responsible for all these files and the previous responsible node should delete these files from its list.

Important Implementation Notice

Please ensure that actual file addition functionality is correctly implemented, as this will be subject to evaluation.

You are not allowed to use **shutil** and **os.makedirs**. Using these functions will result in a **0** for your assignment.

File Placement Rules:

- Each file must physically exist only on its designated node and one backup node.
- Any additional copies result in a score deduction.
- File contents must match the original (verified via checksum).

5.5 Leave

When a node leaves the DHT, it should do so gracefully. This means it should tell its predecessor that it is leaving, communicate with the predecessor, the address of its successor node so the predecessor can update its successor. Moreover, leaving node should also send all its files to the new responsible node. Think about which node will be responsible for these files now.

5.6 Failure Tolerance

As we talked earlier, we need to make our system tolerant to node failures. Failures are commonplace in real life systems, in fact, most designs are made keeping failures in mind as a requirement. We also want our DHT to be resilient to failures. Firstly, you should be able to detect that a node is down, typically this is done by pinging a node. Since we are already pinging the successor, we can check if the successor does not respond for 3 consecutive pings, this means the successor node is down. You will need to keep some state in advance to account for failure. For example, to maintain the ring structure of DHT, every node should keep a list of successors instead of immediate successor. In case the immediate successor is detected to have failed, we can update our immediate successor to the next successor in our list. You

may just keep the state of the second successor instead of keeping a list of successors for this assignment. Next you want to replicate all the files so that even if a node fails, no data is lost. This replication must be done on either the node's successor or the predecessor beforehand but not both of them. Replicating on any other node is not acceptable..

6 Testing

Test cases are run in the order described here. You may not be able to test one part before completing its previous parts as most of the parts have previous parts as their pre-requisite. You will need Python3 to run this file, you can run the tests on any OS but Linux is preferable. There are two ways to test your implementation of the assignment locally. To test your submission, we will run the following command in your assignment directory:

6.1 Individual Tests with `check.py`

To test the DHT on a single port:

```
python3 check.py <port>
```

Example:

```
python3 check.py 10000
```

This runs the test suite for a DHT node starting at port 10000.

Output: Displays detailed test results, including success and error messages.

6.2 Concurrent Tests with `run_multiple_tests.py`

To run tests on multiple ports concurrently:

```
python3 run_multiple_tests.py <port1> <port2> <port3> ...
```

Parameters: `<port1>`, `<port2>`, etc. are ports to start nodes on. If no parameters are passed, the tests run on the default ports.

Example:

```
python3 run_multiple_tests.py 10000 11000 12000
```

You should pass a port between 1000 and 65500. If you start getting an error like:

```
error: [Errno 48] Address already in use
```

Just choose a different port number with a significant gap. Or alternatively, restart the terminal. Remember to test the following ports as they yield noteworthy corner cases: 39580, 59580, 28161, 50396, and 2765.

Important Testing Note

Please be aware that the testing process will involve the use of hidden ports. Hardcoding for specific port numbers is strictly discouraged and may result in incorrect behavior during testing. Ensure that your implementation handles ports programmatically and dynamically.

6.3 Code Quality Check

6.3.1 Overview

`ReviewCodeQuality.py` ensures that the code adheres to standards of:

- Maintainability
- Modularity
- Cyclomatic Complexity
- Comments Documentation

It evaluates files using tools like Radon and Pylint, scoring them on various metrics.

6.3.2 Metrics and Scoring

1. **Maintainability Index (MI)**
   - Grade A/B: 1 point
   - Grade C: 0.75 points
   - Grade D: 0.5 points
   - Grade E: 0.25 points
   - Grade F: 0.1 points
2. **Cyclomatic Complexity (CC)**
   - Grade A/B: 1 point
   - Grade C: 0.75 points
   - Grade D: 0.5 points
   - Grade E: 0.25 points
   - Grade F: 0.1 points
3. **Pylint Score (scaled to 2.5 points)**
   - Score 8–10: 2.5 points
   - Score 7–7.9: 2 points
   - Score 5–6.9: 1.5 points
   - Score 3–4.9: 1 point
   - Below 3: 0.5 points
4. **Comments Ratio (percentage of comments in the code)**
   - >= 15%: 0.5 points
   - 10–14.9%: 0.45 points

- 5–9.9%: 0.35 points
- 2–4.9%: 0.25 points
- <2%: 0.1 points

7 Running the Code on Docker

A Dockerfile is included for consistent testing. Use the following steps:

7.1 Build and Run Container

To build and run the container, use the following command:

```
docker compose run --rm netcen-spring-2026
# or
docker-compose run --rm netcen-spring-2026
```

What this command does:

- `run`: Executes the `netcen-spring-2026` service defined in the `docker-compose.yml` file as a temporary, one-off container.
- `-rm`: Automatically removes the container after the task completes, ensuring no leftover containers clutter the system.
- This command starts the container and runs the service or command specified for `netcen-spring-2026` in `docker-compose.yml`.

7.2 Access Code

- The code is mounted at `/home/netcen_pa4` inside the container.
- This allows you to access and work with your files directly within the container.

Good Luck, and Happy Coding!
A pen and paper will be your best friend for this one, and not Claude/GPT :)